Project Submission–Database Administration

Vehicle Service Management Database

Student Details

Student Name	S.Sarulatha
Register Number	2422J0642
Department	Computer Application
ID	4BA6A21642802B4843C4CD3A99F512FD
College	Tiruppur Kumaran College For Women
Date Of Submission	29/09/2026
Github Link	

1. Problem Statement & –5Marks

Problem Statement

A vehicle service center needs a database to efficiently manage information about customers, their vehicles, and the services performed on those vehicles. Maintaining these details manually can make it difficult to track service history, retrieve customer information, and calculate service-related costs.

Design and implement a Vehicle Service Management Database using MySQL Workbench. The database should store customer details, vehicle information, and service records. Appropriate relationships should be established between the tables, and SQL queries should be used to perform data management and retrieval operations.

Objectives

- To design a simple relational database for a vehicle service center.
- To create and manage tables using MySQL Workbench.
- To establish relationships between related tables.
- To apply primary keys, foreign keys, and other basic constraints.
- To perform data manipulation and retrieval using SQL.
- To use SQL queries to generate useful information from the database.
- To understand the practical application of relational database concepts.

2. DatabaseDesign –10Marks

2.1 Database Structure

The vehicle service management data base is designed to store manage customer,vehicle,and serve information in a structured manner. The database consist of three main tables: customers, vehicles, and service. These tables are connected using primary key and foreign key to maintain data integrity and relationships.

Tables Used

Table	Purpose
Customer	Stores customer information such as customer name, contact number,email, address.
Vehicle	Stores vehicle information such as vehicle number, type, model and the customer associated with the vehicle.
Service	Stores vehicle service records such as service date, service type, service status, and service cost

Relationship:

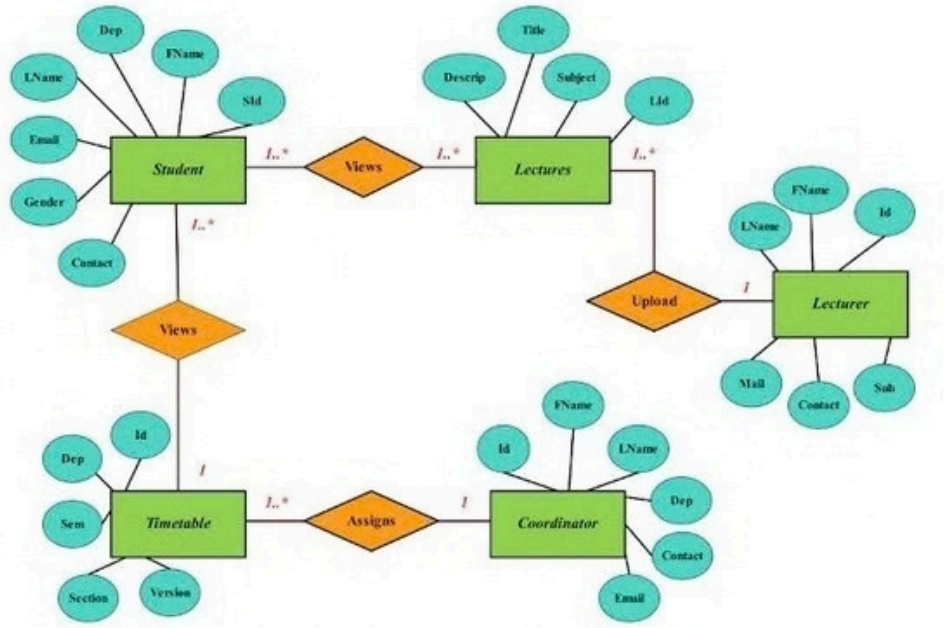
- One customer can have many vehicles.
- One vehicle can have many service records.

This structure helps the service center easily maintain and retrieve customer,vehicle,and service information

2.2 Entity Relationship Diagram

The ER diagram shows the tables, attributes, primary keys, foreign keys, and relationships between the tables.

Diagram:



2.3 Technologies Used

Mentionthetechnologies / tools used and their purpose.

Technology/Tool	Purpose
MySQL	Database management system to store and manage customer, vehicle, and service records
MySQLWorkbench	MySQLworkbench is used to create tables, execute SQL queries, manage the database, and view querie results

3. Practical Implementation –15Marks

3.1Database and Table Creation

Database Creation

```
CREATE DATABASE vehicle_service;
USE vehicle_service;
```

Customer Table

```
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY, customer_name
    VARCHAR(50) NOT NULL, contact VARCHAR(15)
    NOT NULL,
    email VARCHAR(100) UNIQUE,
    address VARCHAR(100)
);
```

Vehicle Table

```
CREATETABLE Vehicle (
    vehicle_idINT PRIMARY KEY,
    vehicle_number VARCHAR(20) NOT NULL UNIQUE,
    vehicle_type VARCHAR(30) NOT NULL,
    vehicle_model VARCHAR(50) NOT NULL,
    customer_id INT NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES
    Customer(customer_id)
     );
```

Service Table

```
CREATE TABLE Service (
    service_id INT PRIMARY KEY,
    vehicle_id INT NOT NULL,
    service_date DATE NOT NULL,
    service_type VARCHAR(50) NOT NULL,
    service_status VARCHAR(30) NOT NULL,
    service_cost DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (vehicle_id) REFERENCES Vehicle(vehicle_id)

);
```

3.2 Constraints and Relationships

Explain the primary keys,foreign keys,and other constraints applied to the tables

Table	Primary	Foreign Key	Other Constraints
Customer	Customer_id	-	NOT NULL,UNIQUE
Vehicle	Vehicle	Customer_id	NOT NULL,UNIQUE
Service	Service_id	Vehicle_id	NOT NULL

- Customer 1: M Vehicle – one customer can own multiple vehicles.
- Vehicle 1: M Service – one vehicle can have multiple service records.

3.3 Data Insertion

Customer Data

```
INSERT INTO Customer VALUES
(1, 'Ravi', '9876543210', 'ravi@gmail.com', 'Chennai'),
(2, 'Priya', '9876501234', 'priya@gmail.com', 'Madurai'),
(3, 'Kumar', '9865432109', 'kumar@gmail.com', 'Coimbatore');
```

Vehicle Data

```
INSERT INTO Vehicle VALUES
(101, 'TN01AB1234', 'Car', 'Swift', 1),
(102, 'TN58CD5678', 'Bike', 'Activa', 2),
(103, 'TN37EF9012', 'Car', 'i20', 3);
```

Service Data

```
INSERT INTO Service VALUES
(1, 101, '2026-09-12', 'Oil Change', 'Completed', 75.50),
(2, 101, '2026-09-14', 'Brake Replacement', 'Pending', 250.00), (3,
102, '2026-09-15', 'Tire Rotation', 'In Progress', 45.00),
(4, 103, '2026-09-16', 'Battery Change', 'Completed', 120.00);
```

3.4 SQL Operations

INSERT

```
INSERT INTO Customer VALUES (4, 'Anu', '9123456780', 'anu@gmail.com',
'Salem');
```

SELECT

```
SELECT * FROM Customer;
```

UPDATE

```
UPDATE Customer SET contact = '9999999999' WHERE customer_id = 1;
```

DELETE

```
DELETE FROM Customer WHERE customer_id = 4;
```

WHERE

```
SELECT * FROM Service WHERE service_status = 'Completed';
```

ORDER BY

```
SELECT * FROM Service ORDER BY service_cost DESC;
```

LIKE

```
SELECT * FROM Vehicle WHERE vehicle_number LIKE 'TN01%';
```

BETWEEN

```
SELECT * FROM Service WHERE service_cost BETWEEN 1000 AND 3000;
```

Aggregate Functions

```
SELECT COUNT(*) AS total_services FROM Service; SELECT
SUM(service_cost) AS total_cost FROM Service; SELECT
AVG(service_cost) AS average_cost FROM Service; SELECT
MAX(service_cost) AS highest_cost FROM Service; SELECT
MIN(service_cost) AS lowest_cost FROM Service;
```

GROUP BY

```
SELECT service_status, COUNT(*) AS service_count
FROM Service
GROUP BY service_status;
```

INNER JOIN

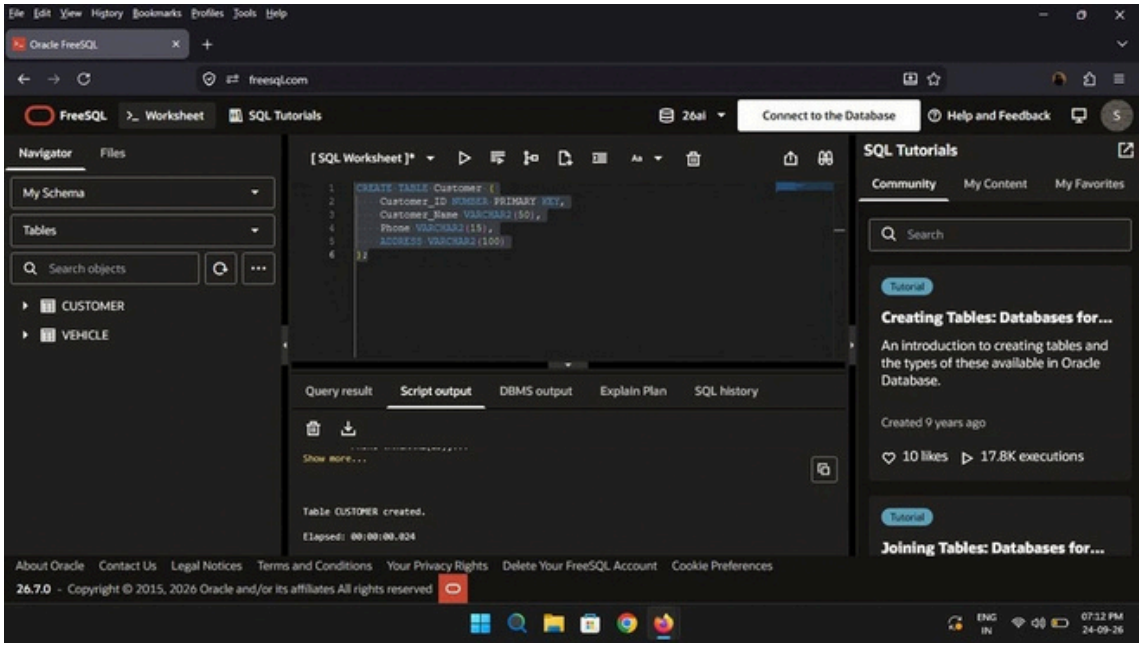
```
SELECT c.customer_name, v.vehicle_number, v.vehicle_model,
    s.service_type, s.service_status, s.service_cost
FROM Customer c
INNER JOIN Vehicle v ON c.customer_id = v.customer_id
INNER JOIN Service s ON v.vehicle_id = s.vehicle_id;
```

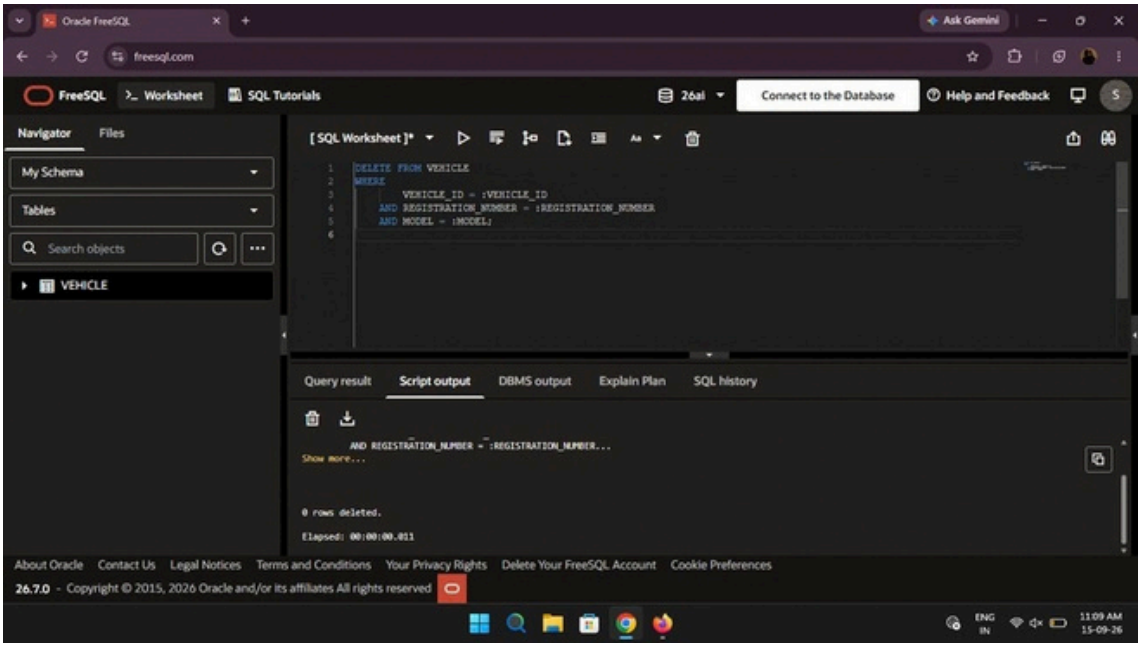
3.5 Project Execution

The database was implemented using MySQL Workbench. First, the database was created using CREATE DATABASE and selected using USE. Then the Customer, Vehicle, and Service tables were created with primary keys, foreign keys, and other constraints. Sample records were inserted using INSERT statements. SELECT, UPDATE, DELETE, WHERE, ORDER BY, LIKE, BETWEEN, aggregate functions, GROUP BY, and INNER JOIN queries were executed to test data manipulation and retrieval. The results were verified in the MySQL Workbench output window.

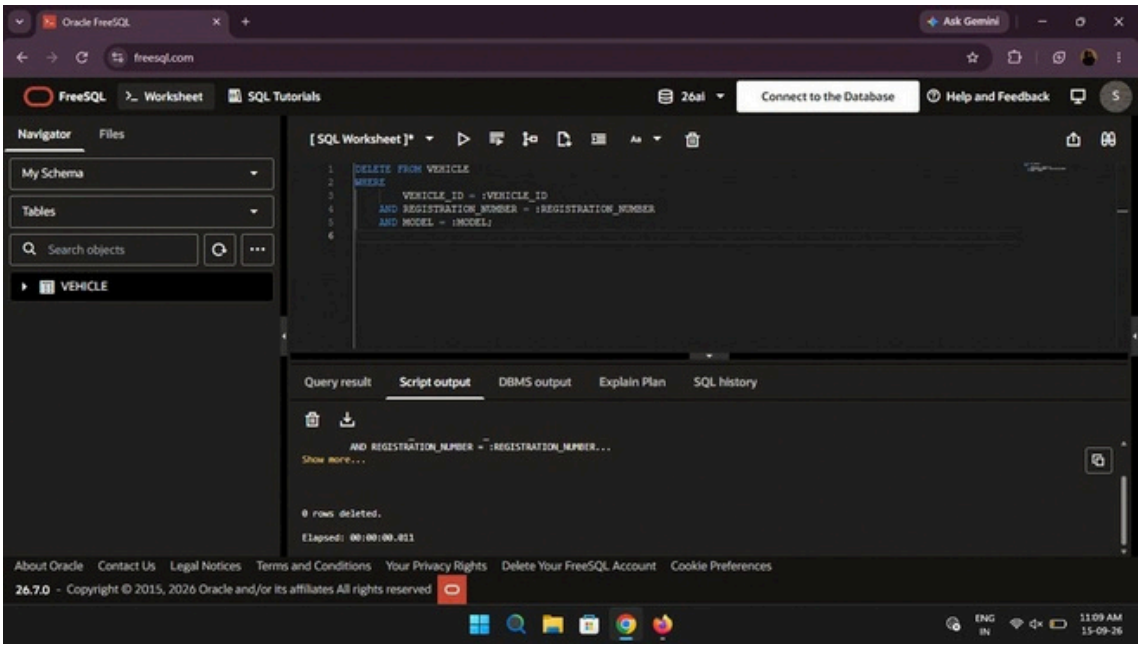
4. Output Screenshots–10Marks

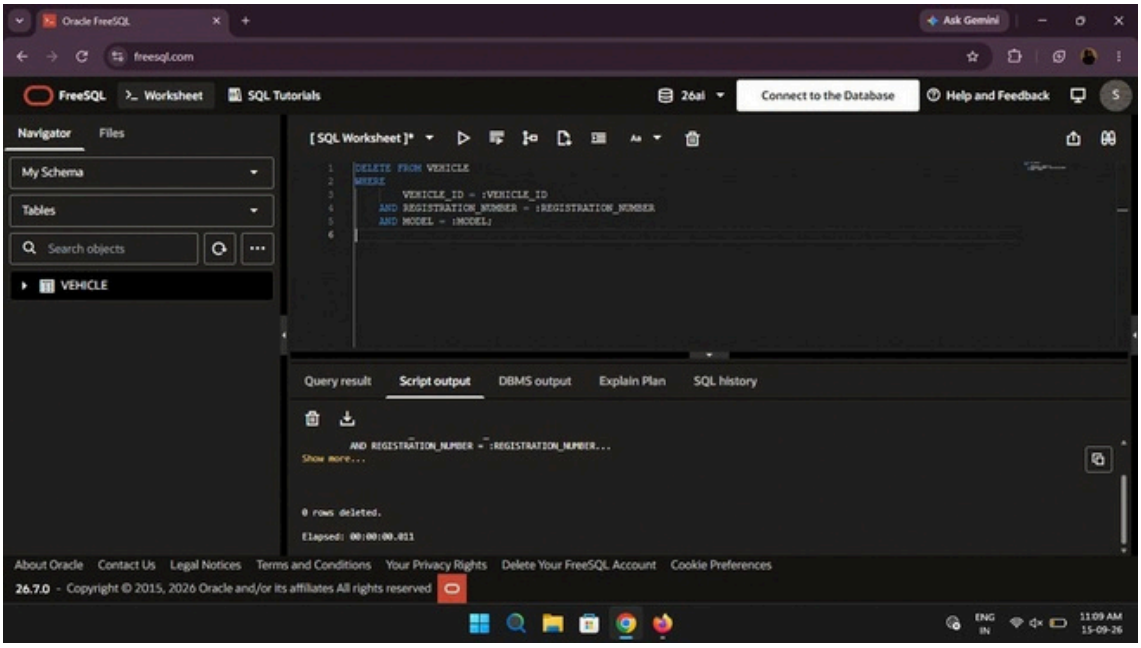
Screenshot 1–Database and Tables



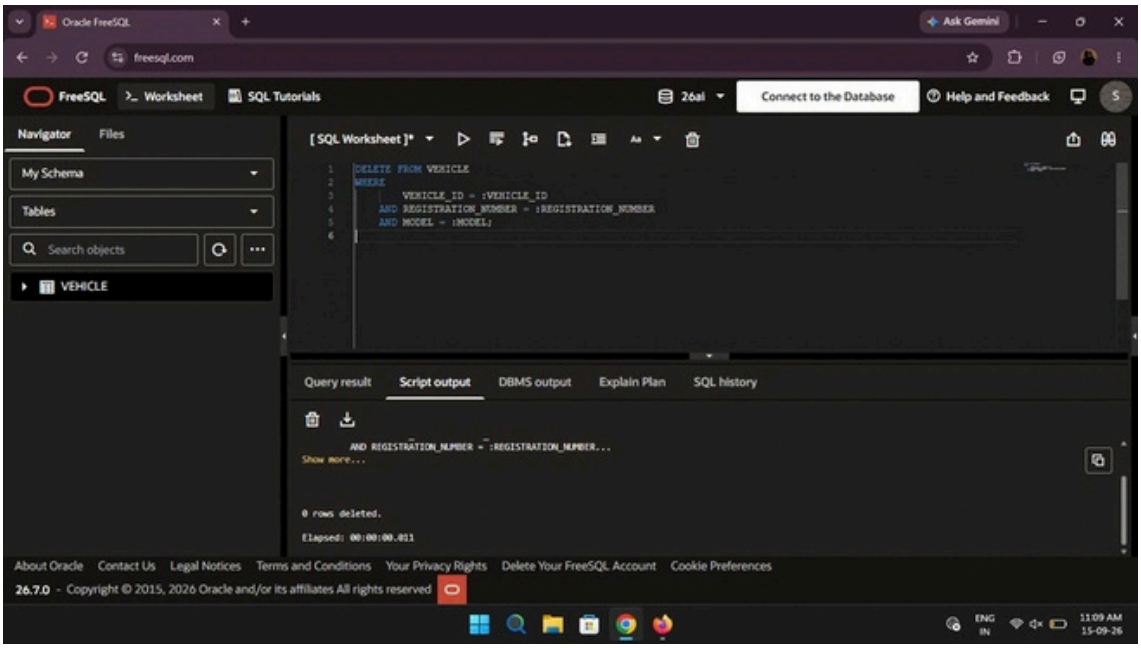


Screenshot 2–Customer Table with Records

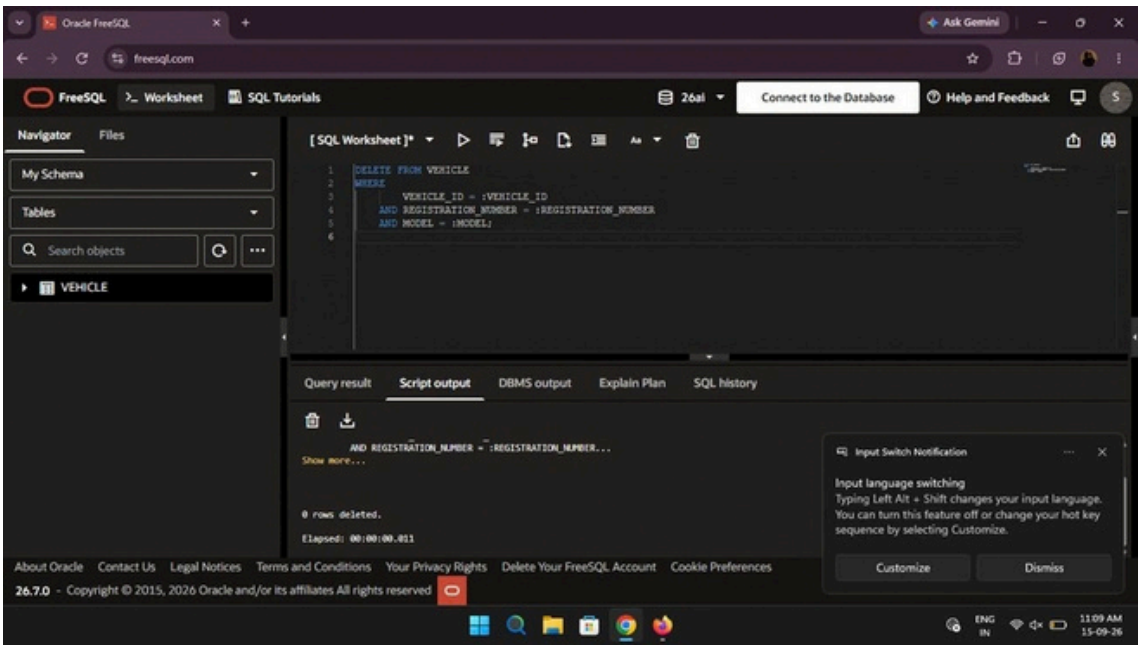




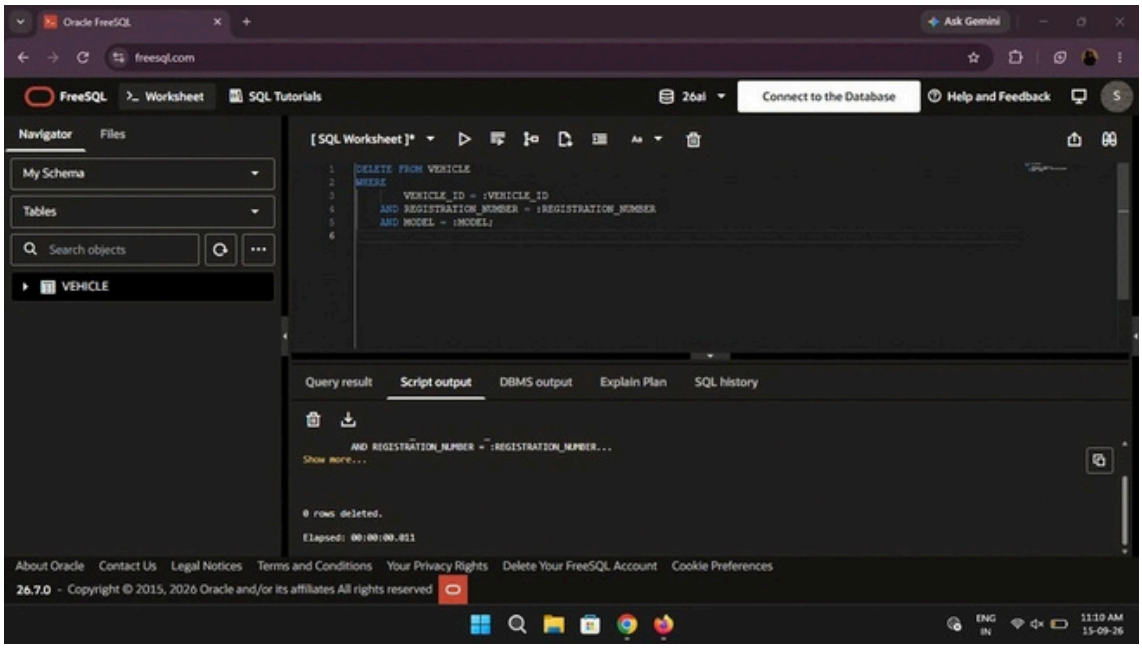
Screenshot 3–Vehicle Table with Records

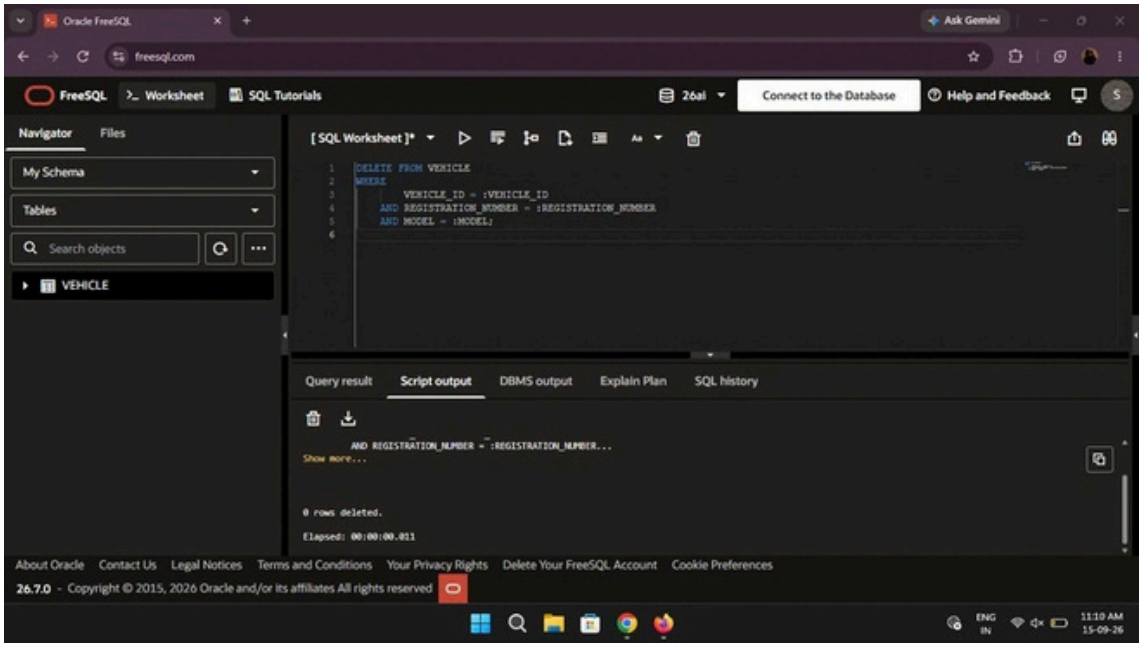


Screenshot 4–Service Table with Records

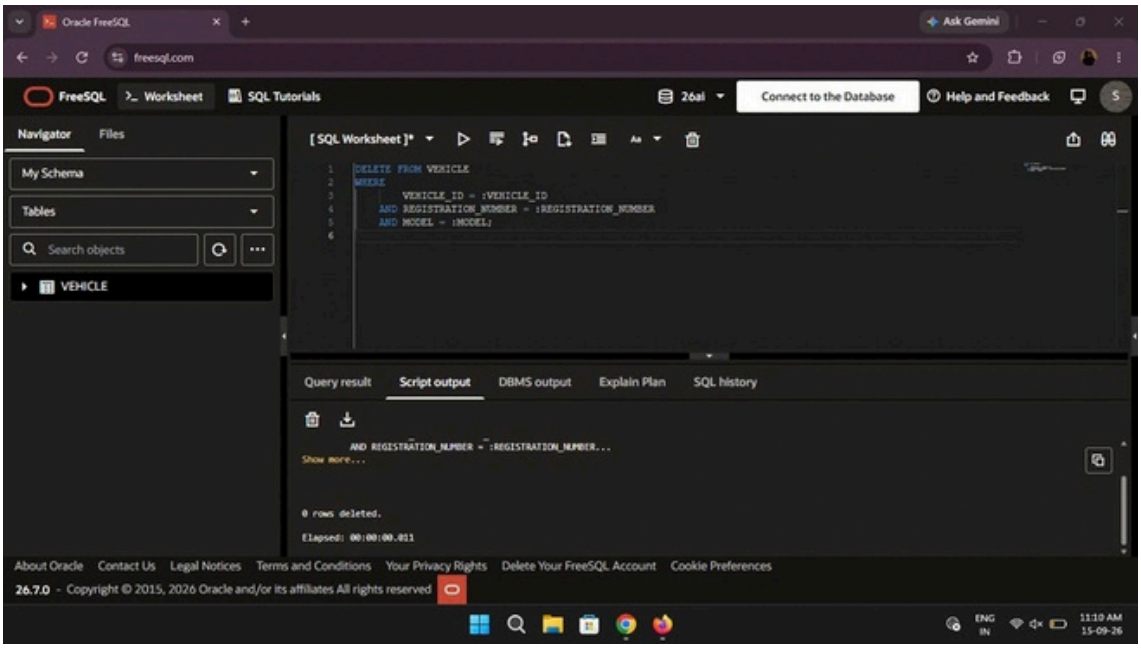


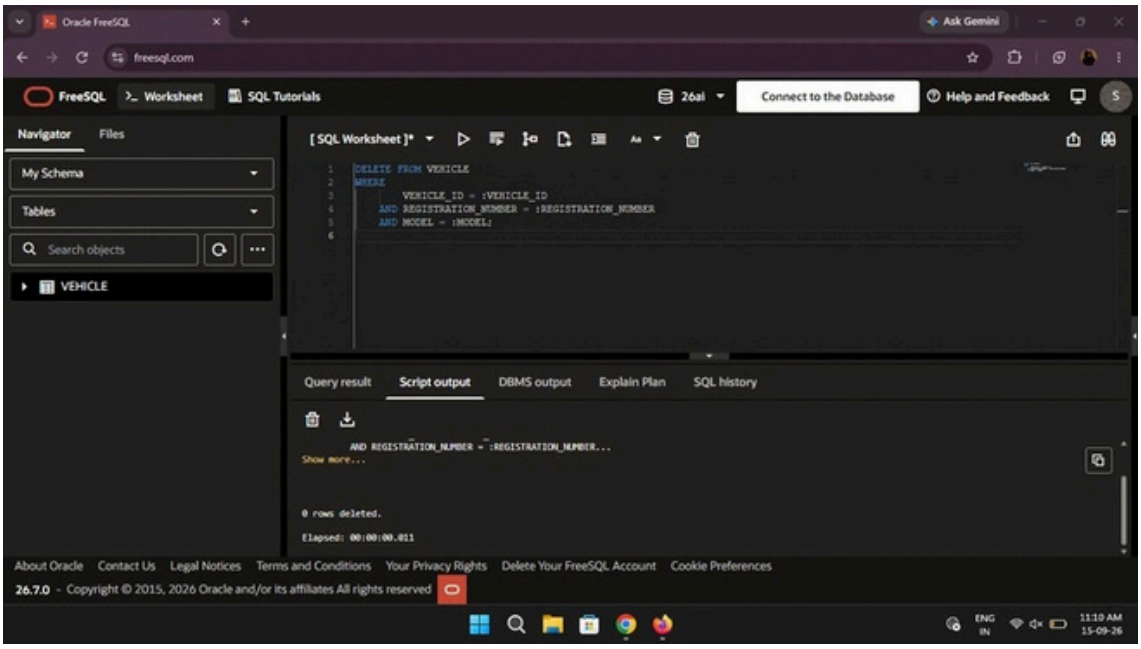
Screenshot 5–INSERT/UPDATE/DELETE Operations



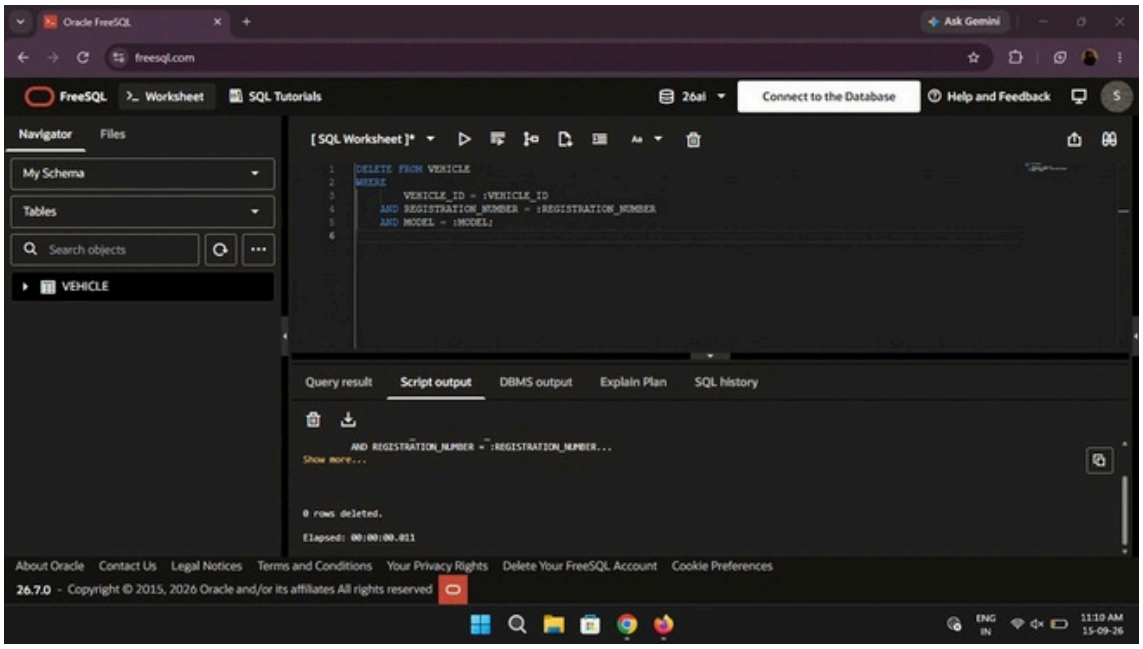


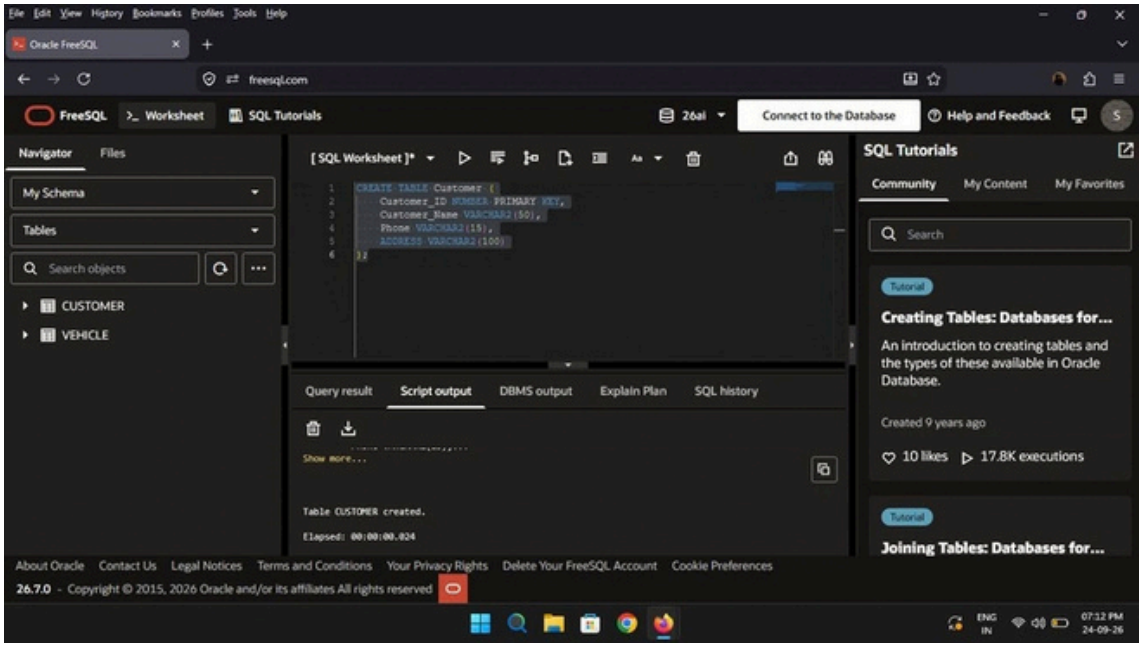
Screenshot 6 –SELECT and Filtering Operations



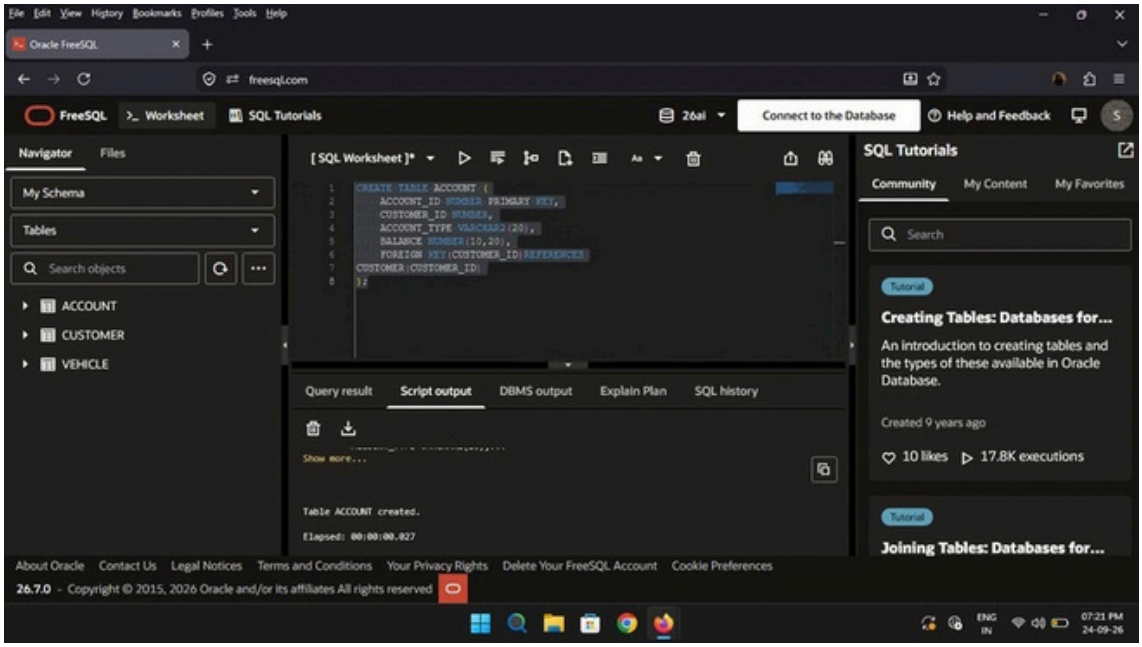


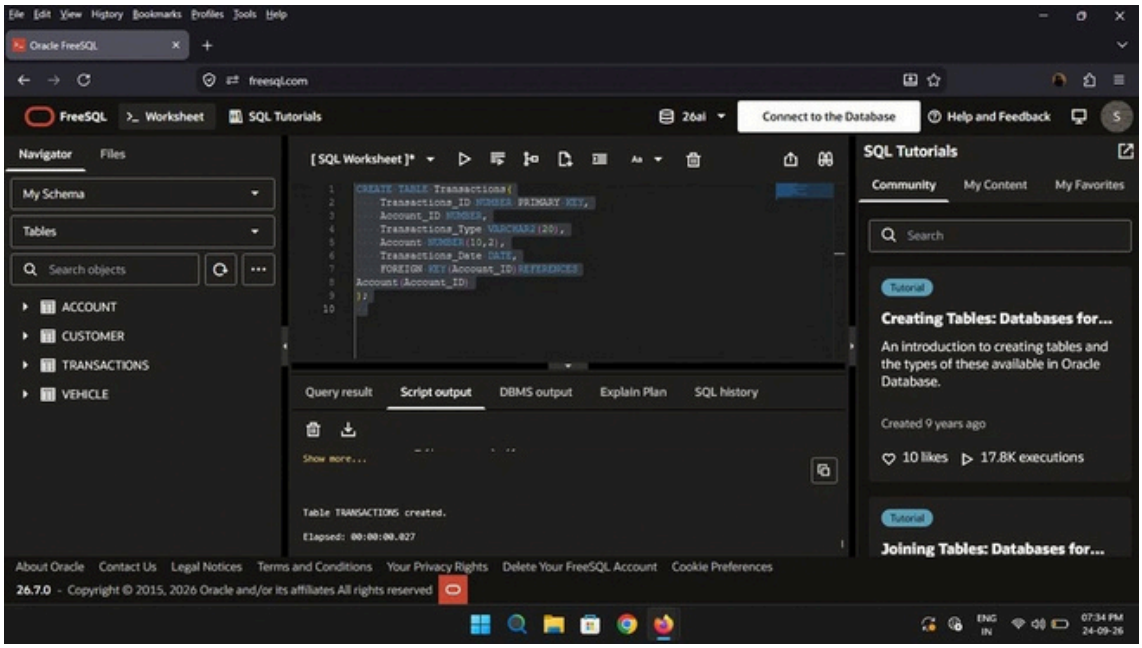
Screenshot 7–Aggregate Functions



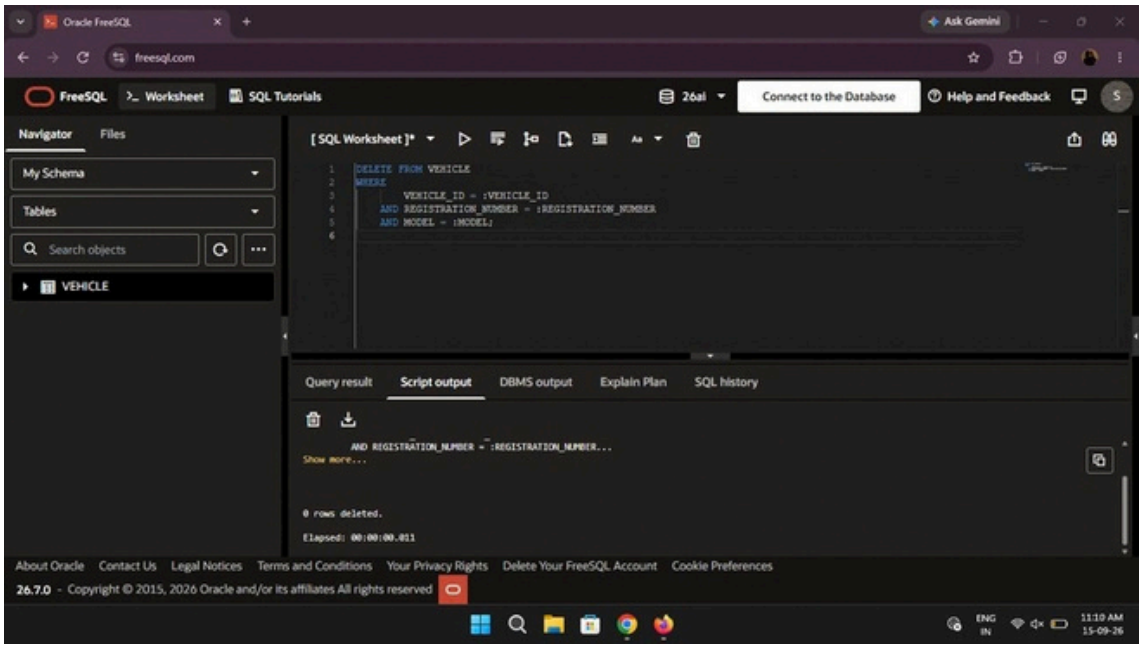


Screenshot 8–GROUP BY Operation

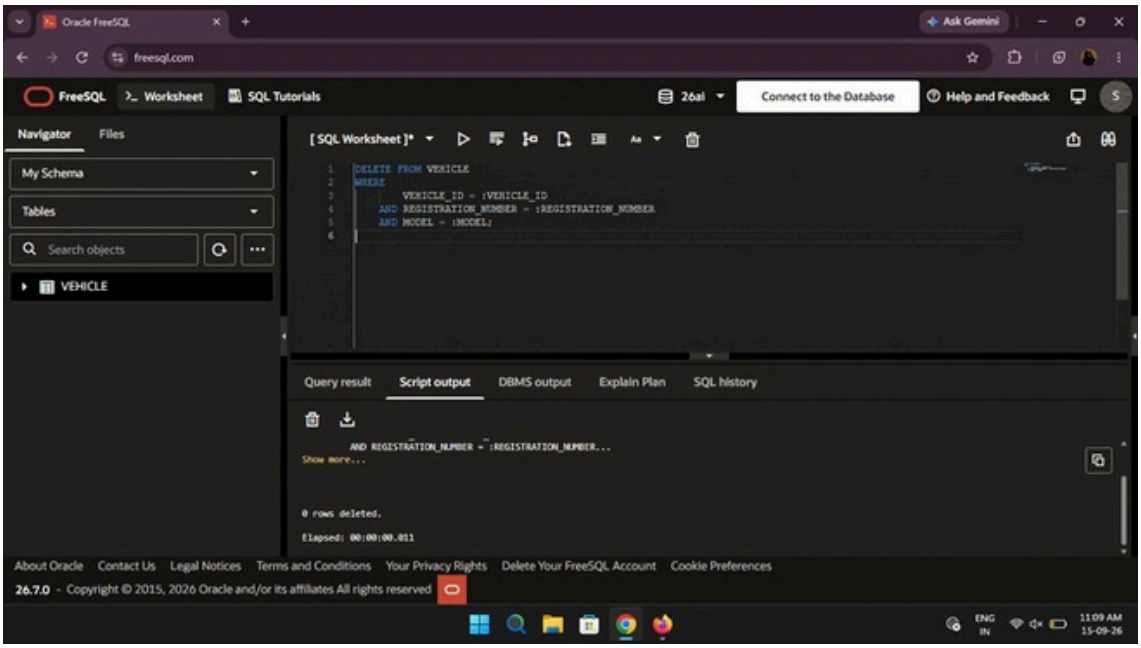




Screenshot 9–INNER JOIN Operation



Screenshot 10 – Final Database/Query Results



5. Technical Explanation–5Marks

Database Design

The database is divided into three tables to avoid unnecessary duplication and to keep the data organized. Customer stores owner details, Vehicle stores vehicle details, and Service stores service history. The tables are connected using foreign keys.

Keys and Constraints

Primary keys uniquely identify records. Foreign keys establish relationships between tables. NOT NULL ensures required values are entered, while UNIQUE prevents duplicate values such as vehicle numbers and email addresses.

SQL Operations

INSERT adds records, SELECT retrieves records, UPDATE modifies existing records, and DELETE removes records. These operations are used to manage the database.

Data Retrieval

WHERE filters records based on a condition, ORDER BY sorts records, LIKE searches for partial values, BETWEEN retrieves values within a range, and GROUP BY groups records according to a selected column.

Aggregate Functions and Joins

COUNT, SUM, AVG, MAX, and MIN perform calculations on groups of records. INNER JOIN combines related information from Customer, Vehicle, and Service tables using their keys.

6. Conclusion–5Marks

Conclusion

The Vehicle Service Management Database successfully manages customer, vehicle, and service information using a relational database model. Primary keys, foreign keys, constraints, SQL operations, filtering, sorting, aggregate functions, GROUP BY, and INNER JOIN were implemented. The project provides practical understanding of database design, data integrity, and SQL-based data management.

Future Enhancements

Mention possible future improvements, such as:

- Customer login and authentication.
- Online service appointment booking
- Automated service reminders.
- Service history reports.
- Invoice generation.
- Mechanic assignment and management.
- Integration with awebor mobile application
- Automated database backup and recovery.